



Estructura Ampliada y Comparativa del Capítulo 6

6.1. Definición del Entorno de Pruebas y Metodología

- **Contenido:** Añadir al escenario físico real la especificación de un **script simulador de carga masiva** (ej. desarrollado en Python con multiprocessing o locust) ejecutado desde un PC para clonar el comportamiento de múltiples nodos IoT concurrentes.
- **Métrica:** Herramientas de telemetría del sistema para la Raspberry Pi 4 (psutil para CPU/RAM, temperatura del SoC) y métricas de rendimiento de la EVM (bloques por segundo, transacciones por minuto).

6.2. Latencia del Canal de Red y Análisis Extremo a Extremo (E2E)

- **Métrica: Latencia Acumulada E2E** en milisegundos. Mide el ciclo de vida completo del dato: desde que el ESP32 genera la lectura de \$eCO_2\$, pasando por el envío CoAP, el procesamiento asíncrono en la Raspberry Pi 4, el consenso en la Blockchain, hasta que un cliente web en el PC lee el dato indexado.
- **Enfoque Comparativo:** No midas solo el RTT de red. Compara la arquitectura propuesta con una arquitectura clásica descentralizada donde cada nodo IoT enviara directamente a la Blockchain mediante HTTP/HTTPS (Edge-to-Cloud directo).
- **Gráfica Recomendada:** Gráfico de áreas apiladas (Stacked Area Chart) que muestre qué porcentaje del tiempo total E2E se pierde en cada etapa (Red, Fog, Blockchain) a medida que aumenta la carga.

6.3. Pruebas de Estrés y Escalabilidad de la Capa Fog ante Carga Concurrente

- **Métrica:** Tasa de éxito de peticiones, tiempo de bloqueo de asyncio y degradación de recursos de la Raspberry Pi 4.
- **Enfoque Comparativo:** Someter a la pasarela a ráfagas crecientes de dispositivos simulados (ej. 10, 50, 100, 500 y 1000 dispositivos IoT virtuales transmitiendo simultáneamente).
- **Gráfica Recomendada:** Gráfico de doble eje Y. El eje izquierdo muestra el uso de CPU/RAM de la RPi 4 y el derecho muestra el porcentaje de datagramas CoAP perdidos o encolados en función del número de dispositivos simulados en paralelo.

6.4. Análisis Estadístico del Consolidador Local (Búfer)

- **Métrica:** Distribución de tiempos de procesamiento locales.
- **Enfoque Comparativo:** Evaluar de forma matemática exacta el impacto de la ventana del búfer comparando los escenarios de 1 (sin consolidación), 5 y 10 muestras.
- **Gráfica Recomendada:** Diagrama de caja y bigotes (*Boxplot*) tal como se definía originalmente, que verifique visualmente la reducción de varianza en el procesamiento del Fog.

6.5. Costes Computacionales y Proyección Temporal del Consumo de Gas

- **Métrica:** Unidades de gas acumuladas de la EVM y coste financiero estimado (en Ether o moneda local según el gas price de la red).
- **Enfoque Comparativo:** Comparar el modelo propuesto (Búfer de 5 muestras con escritura condicional) frente al modelo tradicional de persistencia directa (enviar cada dato individualmente a la blockchain de forma síncrona).
- **Proyección Temporal:** Multiplicar este coste atómico en simulaciones extrapoladas a **1 día, 1 semana y 1 mes completo de operación continua** (asumiendo frecuencias fijas de envío, por ejemplo, cada 10 segundos).

Matriz Comparativa General de los Experimentos

Para facilitar la lectura antes del desarrollo textual, incluye una tabla resumen que deje clara la filosofía comparativa exigida:

Dimensión Analizada	Escenario Base (Tradicional / Sin Optimizar)	Escenario Propuesto (Con Red Fog y Consolidación)	Propósito de la Comparativa
Latencia Temporal	Envío directo Edge-to-Blockchain vía HTTPS / Web3 RPC.	Tránsito CoAP hacia Fog + Consolidación local asíncrona (p. 1).	Demostrar la viabilidad en tiempo real reduciendo el cuello de botella del bloque de la EVM.
Capacidad de Carga	Conexiones TCP individuales directas saturando los sockets de red.	Simulación de \$N\$ nodos concurrentes canalizados a través de un único broker local en la RPi4.	Evaluar el punto de ruptura del hardware Fog y la Blockchain ante ráfagas masivas.
Eficiencia Económica	Envío individual por cada muestra generada (\$N\$ transacciones \$SSTORE\$).	Envío agrupado cada 5 muestras (\$1\$ transacción compartida) (p. 1).	Evidenciar el ahorro porcentual real de gas en periodos prolongados de tiempo (Día/Semana) (p. 2).



Código Base Matplotlib para la Proyección Temporal de Gas (Sección 6.5)

Para que puedas generar la gráfica de proyección temporal comparativa de forma idéntica a la estética académica anterior (p. 2), utiliza este script de Python adaptado:

```
import matplotlib.pyplot as plt
import numpy as np

# Configuración de horizontes temporales y frecuencias
# Asumiendo 1 dato cada 10 segundos por dispositivo -> 6 datos por minuto -> 360 por hora
datos_dia = 360 * 24
periodos = ['1 Día', '1 Semana (7d)', '1 Mes (30d)']

# Consumo de gas estimado (Unidades de gas de la EVM)
# Tradicional: Cada dato gasta ~45,000 gas (operación individual de escritura)
# Propuesto: Almacena 4 lecturas intermedias en memoria interna (~0 gas en blockchain)
# y la 5ª ejecuta el consenso real (~85,000 gas) -> Coste medio por dato = 85,000 / 5 = 17,000 gas
gas_tradicional = np.array([datos_dia * 45000, datos_dia * 7 * 45000, datos_dia * 30 * 45000])
gas_propuesto = np.array([(datos_dia / 5) * 85000, ((datos_dia * 7) / 5) * 85000, ((datos_dia * 30) / 5) * 85000])

# Configuración estética académica
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 11

x = np.arange(len(periodos))
width = 0.35

fig, ax = plt.subplots(figsize=(8, 5))
rects1 = ax.bar(x - width/2, gas_tradicional, width, label='Enfoque Tradicional (Sin Búfer)', color='#B22222', edgecolor='black')
rects2 = ax.bar(x + width/2, gas_propuesto, width, label='Enfoque Propuesto (Búfer Con Fog)', color='#2E8B57', edgecolor='black')

# Etiquetas y títulos
ax.set_ylabel('Consumo Total de Gas Acumulado (EVM)', fontsize=12, fontweight='bold')
ax.set_title('Proyección Comparativa de Costes de Gas a Largo Plazo', fontsize=13, fontweight='bold', pad=15)
ax.set_xticks(x)
```

```

ax.set_xticklabels(periodos)
ax.legend(loc='upper left')
ax.grid(axis='y', linestyle='--', alpha=0.5)

# Formatear números grandes en el eje Y
ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, loc: "{:,}".format(int(x))))

# Añadir etiquetas de porcentaje de ahorro sobre las barras propuestas
for i in range(len(periodos)):
    ahorro = ((gas_tradicional[i] - gas_propuesto[i]) / gas_tradicional[i]) * 100
    ax.annotate(f'-{ahorro:.1f}% coste',
                xy=(x[i] + width/2, gas_propuesto[i]),
                xytext=(0, 5),
                textcoords="offset points",
                ha='center', va='bottom', fontsize=9, fontweight='bold', color='#1E5631')

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig('proyeccion_temporal_gas.png', dpi=300)
plt.show()

```

Para avanzar con el desarrollo de los scripts de pruebas, dime:

- ¿Qué **blockchain / simulador** concreto utilizas? (¿Hardhat local, Ganache, una red de pruebas como Sepolia?)
- ¿Cuál es la **frecuencia de muestreo exacta** planeada para tus sensores reales (cada cuánto envían datos)?

Con estos datos puedo ajustarte las fórmulas de latencia y costes de forma matemáticamente exacta para tu memoria.